

Motor de Inferencia Lógica como Servicio

Reporte Técnico del Proyecto

Asignatura: Teoría de Lenguajes de Programación

Fecha: Mayo 2026

1. Introducción

El presente proyecto tiene como objetivo el diseño e implementación de un Motor de Inferencia Lógica como Servicio. Esta solución tecnológica integra un motor de razonamiento simbólico basado en lógica declarativa (Prolog) con una arquitectura de servicios web moderna, desarrollada mediante Node.js y el framework Express.

El sistema permite a los clientes externos ejecutar consultas lógicas sobre una base de conocimiento dinámica a través de una API RESTful. Al recibir una petición HTTP, el servidor delega la resolución del problema al motor lógico, el cual aplica reglas de inferencia para derivar conclusiones automáticas basadas en hechos predefinidos.

La relevancia de este proyecto radica en su capacidad para demostrar la viabilidad de integrar paradigmas de programación tradicionalmente dispares —la programación lógica simbólica y la programación de servidores asíncrona moderna— dentro de una misma arquitectura de software cohesiva, escalable y eficiente.

2. Explicación de Paradigmas de Programación Utilizados

El éxito de la arquitectura reside en la selección estratégica de paradigmas de programación, aprovechando las fortalezas específicas de cada uno para resolver sub-problemas distintos dentro del sistema.

2.1 Programación Lógica (Prolog)

Este paradigma constituye el núcleo intelectual del sistema. Se implementa mediante el lenguaje Prolog, el cual se basa en la definición de hechos (datos atómicos) y reglas (relaciones lógicas).

Función: Representar el dominio del problema y realizar inferencias automáticas mediante un mecanismo de unificación y retroceso (backtracking).

Implementación: El sistema evalúa consultas como `penalty_applicable(X)` contra una base de conocimientos estática, determinando la veracidad de proposiciones sin necesidad de algoritmos imperativos explícitos.

```
% Ejemplo de base de conocimiento
contract(contract1).
breach(contract1).
penalty_applicable(X) :- contract(X), breach(X).
```

2.2 Programación Funcional (JavaScript)

En la capa de aplicación (Node.js), se adoptan principios de la programación funcional para el procesamiento de datos.

Función: Transformación de datos, normalización de entradas y composición de funciones puras.

Beneficios:

Modularidad: Código dividido en funciones pequeñas y reutilizables.

Previsibilidad: Reducción de efectos secundarios (side effects), facilitando las pruebas unitarias.

Separación de responsabilidades: Aislamiento claro entre la lógica de negocio y la manipulación de datos.

2.3 Programación Asíncrona (Node.js)

Dado que el sistema opera como un servicio web, la gestión de la concurrencia es crítica. Node.js utiliza un modelo de bucle de eventos (event loop) y operaciones no bloqueantes.

Función: Manejo eficiente de múltiples solicitudes HTTP simultáneas sin bloquear el hilo principal.

Herramientas: Uso intensivo de Promises y sintaxis async/await para gestionar el flujo de control durante la interacción con el motor Prolog y la red.

2.4 Síntesis de Paradigmas

Paradigma	Rol en el Sistema	Tecnología
Lógico	Inferencia y razonamiento simbólico	Prolog (Tau Prolog)
Funcional	Transformación y flujo de datos	JavaScript (ES6+)
Asíncrono	Concurrencia y manejo de I/O	Node.js Event Loop
Modular	Estructura y mantenibilidad	Arquitectura de Módulos

3. Arquitectura del Sistema

El sistema sigue un patrón de arquitectura en capas, diseñado para mantener una clara separación de preocupaciones (Separation of Concerns) y facilitar la escalabilidad futura.

3.1 Flujo de Datos General

Cliente: Envía una petición HTTP POST con una consulta lógica en formato JSON.

Capa de Transporte (Express): Recibe la solicitud y la enruta al controlador adecuado.

Capa de Control: Valida la entrada, sanitiza los datos y prepara el contexto.

Capa de Servicio (Lógica): Invoca al motor Prolog, cargando la base de conocimiento si es necesario.

Motor de Inferencia: Ejecuta la consulta, realiza la unificación y devuelve el resultado booleano o las variables instanciadas.

Respuesta: El servidor formatea el resultado y lo devuelve al cliente en formato JSON.

3.2 Estructura del Proyecto

La organización de archivos refleja la modularidad del sistema:

```
src/
├── app.js           # Punto de entrada y configuración de Express
├── routes/
│   └── query.routes.js  # Definición de endpoints de la API
├── controllers/
│   └── query.controller.js # Lógica de control de peticiones HTTP
├── services/
│   └── prolog.service.js  # Integración con Tau Prolog y ejecución de consultas
└── prolog/
```

```
| └─ knowledgeBase.pl  # Base de conocimiento (Hechos y Reglas)
| └─ utils/           # Funciones auxiliares y helpers
```

Descripción de Componentes Clave

app.js: Inicializa el servidor Express, configura el middleware (como body-parser) y monta las rutas.
routes/: Define la superficie de la API, mapeando el método POST en /query al controlador correspondiente.

controllers/: Actúa como intermediario, extrayendo la consulta del cuerpo de la petición y gestionando los errores de entrada.

services/prolog.service.js: Contiene la lógica de negocio crítica. Instancia el motor Tau Prolog, carga el archivo .pl y ejecuta la función de consulta de forma asíncrona.

prolog/knowledgeBase.pl: Archivo fuente que contiene la representación declarativa del dominio del problema.

4. Ejemplo de Ejecución

A continuación, se detalla un caso de uso práctico donde el sistema determina si una penalización es aplicable a un contrato específico.

4.1 Base de Conocimiento (knowledgeBase.pl)

% Hechos

contract(contract1).

breach(contract1).

% Reglas

penalty_applicable(X) :- contract(X), breach(X).

4.2 Solicitud HTTP (Request)

El cliente envía la siguiente petición al endpoint /query:

Método: POST

URL: http://localhost:3000/query

Headers: Content-Type: application/json

Body:

```
{
  "query": "penalty_applicable(contract1)."
}
```

4.3 Proceso de Ejecución

El servidor recibe la petición JSON.

El controlador extrae el string "penalty_applicable(contract1).".

El servicio Prolog carga la base de conocimientos.

El motor intenta satisfacer el predicado penalty_applicable(contract1).

El motor verifica: ¿Existe contract(contract1)? Sí. ¿Existe breach(contract1)? Sí.

La regla se cumple y la inferencia es verdadera.

4.4 Respuesta del Sistema (Response)

```
{  
  "success": true,  
  "result": true,  
  "message": "Inferencia completada exitosamente."  
}
```

5. Conclusiones y Extensiones Futuras

5.1 Conclusiones

El desarrollo de este proyecto ha demostrado que es posible integrar exitosamente la programación lógica, tradicionalmente asociada a la inteligencia artificial y la investigación académica, con las tecnologías estándar de la industria del desarrollo web moderno (Node.js y Express).

Se ha evidenciado que:

La programación lógica ofrece una elegancia superior para problemas que involucran reglas complejas y razonamiento deductivo, reduciendo drásticamente la cantidad de código imperativo necesario.

La programación asíncrona de Node.js es fundamental para exponer estas capacidades como un servicio web de alto rendimiento, capaz de manejar concurrencia sin sacrificar la integridad de las inferencias.

La arquitectura modular permite que el motor lógico evolucione independientemente de la capa de transporte, facilitando el mantenimiento y la escalabilidad.

5.2 Extensiones Futuras

Para llevar este prototipo a un entorno de producción robusto, se proponen las siguientes mejoras:

Multi-tenencia y Bases de Conocimiento Dinámicas: Implementar un sistema que permita cargar y gestionar múltiples archivos .pl de forma dinámica según el usuario o el contexto de la solicitud.

Persistencia de Datos: Integrar una base de datos (SQL o NoSQL) para almacenar hechos históricos y reglas persistentes, evitando recargar archivos estáticos en cada petición.

Interfaz Gráfica de Usuario (GUI): Desarrollar un frontend (React/Vue) que permita a usuarios no técnicos visualizar la base de conocimientos, editar reglas y ejecutar consultas mediante una interfaz amigable.

Autenticación y Autorización: Implementar mecanismos de seguridad (JWT, OAuth) para proteger la API y controlar el acceso a diferentes niveles de conocimiento.

Visualización de Grafos de Inferencia: Generar representaciones gráficas del árbol de deducción utilizado por Prolog para explicar por qué se llegó a una conclusión específica (explicabilidad de la IA).

Optimización de Consultas: Implementar un sistema de caché para consultas frecuentes y optimizar la carga de la base de conocimientos en memoria.

Referencias Bibliográficas

- Bratko, I. (2011). Prolog Programming for Artificial Intelligence. Addison-Wesley.
- Richardson, L., & Ruby, S. (2007). RESTful Web Services. O'Reilly Media.
- Node.js Foundation. (2024). Node.js Documentation. Recuperado de <https://nodejs.org/docs/>
- Strong, P. (2024). Tau Prolog: A Pure Prolog Interpreter in JavaScript. Recuperado de <https://tauprolog.org/>
- Express.js Team. (2024). Express.js Guide. Recuperado de <https://expressjs.com/>